

INDEPENDENT SPACE AI RESEARCH

Kolkata, India

A Two-Stage Deep Learning Framework for Automated Solar Filament Segmentation from Synoptic H α Observations

*A Solution to the Solar Filament Segmentation Challenge 2026***Sourab Ghosh**

sourab.ghosh@example.com

Abstract

This report describes our complete solution to the Solar Filament Segmentation Challenge 2026, a Kaggle competition on automatic instance segmentation of solar filaments in GONG H α synoptic observations. Accurate delineation of filament morphology is essential for understanding coronal magnetic topology and forecasting space-weather events such as Coronal Mass Ejections (CMEs). High variability in filament morphology, delicate barb structures, and observational noise in ground-based imagery make this a challenging segmentation problem. We present a two-stage deep learning framework tailored to the MAGFiLO 1.0 benchmark: a high-resolution YOLO11m detector for candidate localization, followed by a UNet++ segmenter with an EfficientNet-B4 encoder operating on contextually padded crops (512×512). To address severe foreground-background imbalance and to penalize false negatives along thin filament boundaries, the segmenter is trained with a composite Binary Cross-Entropy and Tversky loss ($\alpha = 0.3$, $\beta = 0.7$). A confidence-ranked greedy panoptic painter enforces strictly disjoint instance masks, as required by the competition. On the validation split, the pipeline attains a Panoptic Quality (PQ) of 0.4145 (Detection Quality 0.6093, Segmentation Quality 0.6802, crop Dice 0.8433), yielding a public leaderboard score of 0.3700 and surpassing the reported human inter-annotator agreement benchmark (≈ 0.36).

1. Introduction

Solar filaments — termed *prominences* when observed in projection against the solar limb — are dense, cool plasma structures suspended within magnetic flux ropes above photospheric polarity inversion lines in the solar corona [1]. The destabilization of these magnetic structures is a well-established precursor to Coronal Mass Ejections (CMEs) and solar flares, which in turn can induce severe geomagnetic storms at Earth. Such storms pose material risk to satellite operations, high-frequency radio communication, GPS positioning, and terrestrial power grid infrastructure.

Automated delineation of solar filaments from ground-based synoptic observations — most notably from the Global Oscillation Network Group (GONG) H α (656.3 nm) telescope network — is therefore an important capability for operational space-weather forecasting pipelines. Manual annotation of filament boundaries does not scale to the cadence at which GONG stations acquire full-disk imagery, motivating the development of automated computer-vision systems.

Despite the apparent simplicity of the task as a binary foreground segmentation problem, automated filament segmentation is complicated by several domain-specific factors:

1. **Fine-scale barb delineation:** Filaments possess extended central spines together with thin, low-contrast lateral “barbs” that are easily lost to standard segmentation losses tuned for compact objects.

2. **Observational artifacts:** Ground-based synoptic observations from geographically distributed stations exhibit station-dependent atmospheric turbulence (seeing), sensor noise, and non-uniform illumination arising from limb darkening.
3. **Fragmentation and over-merging:** Splitting a single physical filament into multiple predicted instances (one-to-many error) or merging adjacent, distinct filaments into a single instance (many-to-one error) is heavily penalized under panoptic-style evaluation.
4. **Multi-annotator variance:** Filament boundaries are inherently diffuse rather than sharply defined, which is reflected in a reported human inter-annotator agreement of only ≈ 0.36 [1].

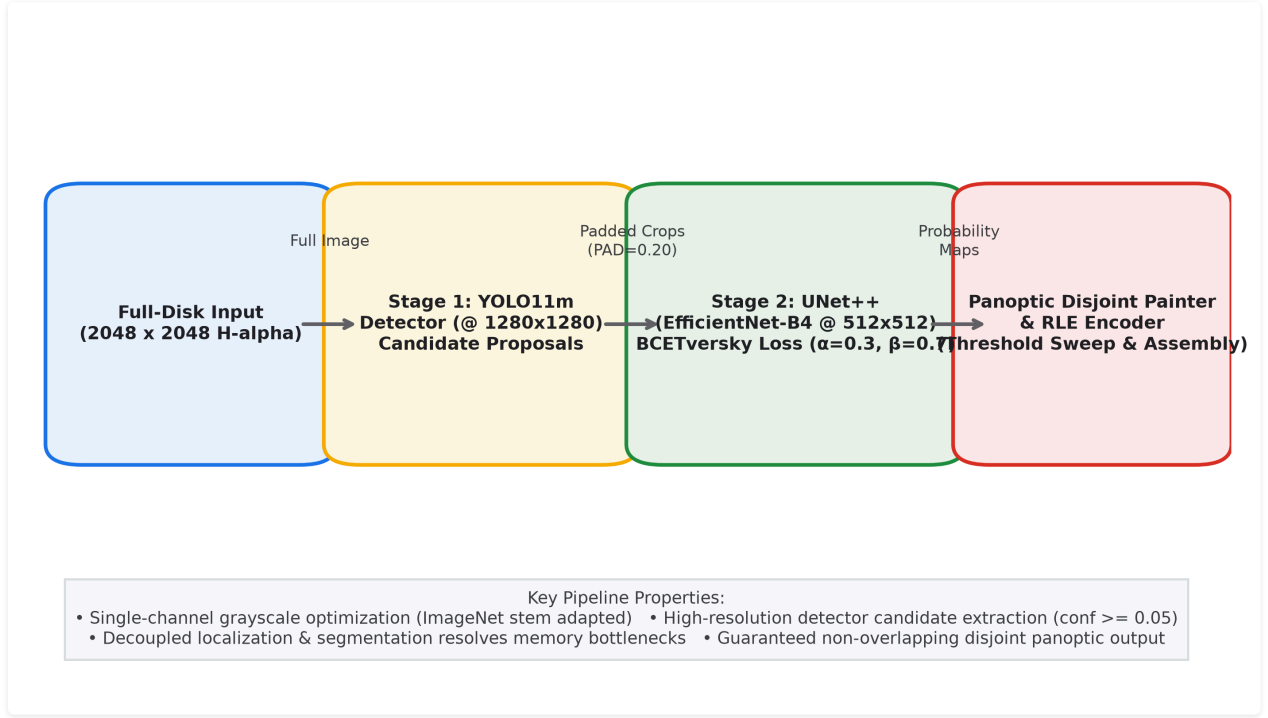


Figure 1: Architectural workflow of the proposed Two-Stage Deep Learning Framework, decoupling candidate proposal (YOLO11m) from high-resolution fine barb segmentation (UNet++ EfficientNet-B4) and greedy panoptic post-processing.

2. Problem Formulation and Evaluation Metric

2.1 Dataset

The MAGFiLO 1.0 dataset [1, 2] comprises 2048×2048 single-channel grayscale GONG H α observations. The training split contains 707 unique full-disk images accompanied by 976 multi-annotator record sets, jointly comprising 8,199 individually labeled filament instances. An additional 180 unannotated observations constitute the held-out competition test set.

2.2 Panoptic Quality Metric

The official competition evaluation criterion is Panoptic Quality (PQ) [4], which decomposes multiplicatively into a Detection Quality (DQ) term and a Segmentation Quality (SQ) term:

$$PQ = DQ \times SQ = \left[|TP| / (|TP| + 0.5|FP| + 0.5|FN|) \right] \times \left[\sum_{(p,g) \in TP} IoU(p, g) / |TP| \right] \quad (1)$$

where p and g denote a predicted instance mask and its matched ground-truth instance, respectively. A predicted-ground-truth pair is declared a True Positive (TP) if and only if $IoU(p, g) > 0.5$; unmatched predictions and

unmatched ground-truth instances are counted as False Positives (FP) and False Negatives (FN) respectively. Critically, the competition imposes a strict disjointness constraint on the predicted instance set:

$$p_i \cap p_j = \emptyset \quad \forall i \neq j \quad (2)$$

3. Proposed Methodology & Implementation

Our framework decouples the segmentation task into two conditionally independent sub-problems: (i) coarse, image-level candidate localization, and (ii) fine-grained, crop-level mask refinement. Below are the key implementation blocks directly from our Jupyter Notebook pipeline (`sfscs.ipynb`).

3.1 Configuration & Stage 1: Candidate Detector (YOLO11m)

Candidate filament regions are localized across the full 2048×2048 solar disk using the Ultralytics YOLO11m single-stage detector [3]. The detector is trained at an input resolution of 1280×1280 pixels using the AdamW optimizer ($\text{lr}_0 = 8.7 \times 10^{-4}$) for 40 epochs. On the validation split, the trained detector attains a mAP_{50} of 0.6369, mAP_{50-95} of 0.3760, precision of 0.6154, and recall of 0.6346.

```
class CFG:
    SEED = 42
    # ----- Stage 1: Detector -----
    DETECTOR      = 'yolo11m.pt'    # YOLO11m detector
    DET_IMGSZ     = 1280
    DET_BATCH     = 8
    DET_EPOCHS    = 40
    DET_PATIENCE  = 8
    DET_LR        = 0.00087

    # ----- Stage 2: Crop Segmenter -----
    ENCODER       = 'efficientnet-b4'
    CROP_SIZE     = 512
    PAD_RATIO     = 0.20            # 20% margin around bounding box
    BOX_JITTER    = 0.00
    SEG_EPOCHS    = 12
    SEG_BATCH     = 8
    SEG_LR        = 2e-4
    TVERSKY_ALPHA, TVERSKY_BETA = 0.3, 0.7

    # ----- Inference -----
    INFER_IMGSZ   = 1024
    CONF_MIN      = 0.05
    MAX_DET       = 60
    MIN_AREA      = 0
    CONF_GRID     = [0.10, 0.15, 0.20, 0.25, 0.30, 0.35]
    MASK_GRID     = [0.40, 0.50, 0.60]
```

Listing 1: Pipeline hyperparameter configuration from `sfscs.ipynb`.

3.2 Stage 2: Crop Segmenter & Composite BCETversky Loss

Each detected bounding box is expanded by a contextual margin of $\text{PAD_RATIO} = 0.20$ prior to cropping, ensuring that thin boundary threads extending beyond the raw box remain within view:

$$\begin{aligned} x_1 &= \max(0, x - 0.20w), & x_2 &= \min(W, x + 1.20w) \\ y_1 &= \max(0, y - 0.20h), & y_2 &= \min(H, y + 1.20h) \end{aligned} \quad (3)$$

To counteract class imbalance and explicitly favor recall on thin barbs, the segmenter is trained with a composite loss combining Binary Cross-Entropy (BCE) and the Tversky loss ($\alpha = 0.3$, $\beta = 0.7$) [5]:

$$L_{total} = 0.5 \cdot L_{BCE} + 0.5 \cdot L_{Tversky} \quad \text{where} \quad L_{Tversky} = 1 - [(TP + \epsilon) / (TP + \alpha FP + \beta FN + \epsilon)] \quad (4)$$

```
class TverskyLoss(nn.Module):
    # beta > alpha penalizes false negatives more: targets missing barbs/filaments
    def __init__(self, alpha=CFG.TVERSKEY_ALPHA, beta=CFG.TVERSKEY_BETA, smooth=1e-6):
        super().__init__()
        self.alpha, self.beta, self.smooth = alpha, beta, smooth

    def forward(self, logits, targets):
        p = torch.sigmoid(logits).view(-1); t = targets.view(-1)
        tp = (p * t).sum(); fp = (p * (1 - t)).sum(); fn = ((1 - p) * t).sum()
        return 1 - (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn + self.smooth)

class BCETverskyLoss(nn.Module):
    def __init__(self):
        super().__init__()
        self.bce, self.tv = nn.BCEWithLogitsLoss(), TverskyLoss()

    def forward(self, logits, targets):
        return 0.5 * self.bce(logits, targets) + 0.5 * self.tv(logits, targets)
```

Listing 2: PyTorch implementation of the composite BCETverskyLoss from `s4scs.ipynb`.

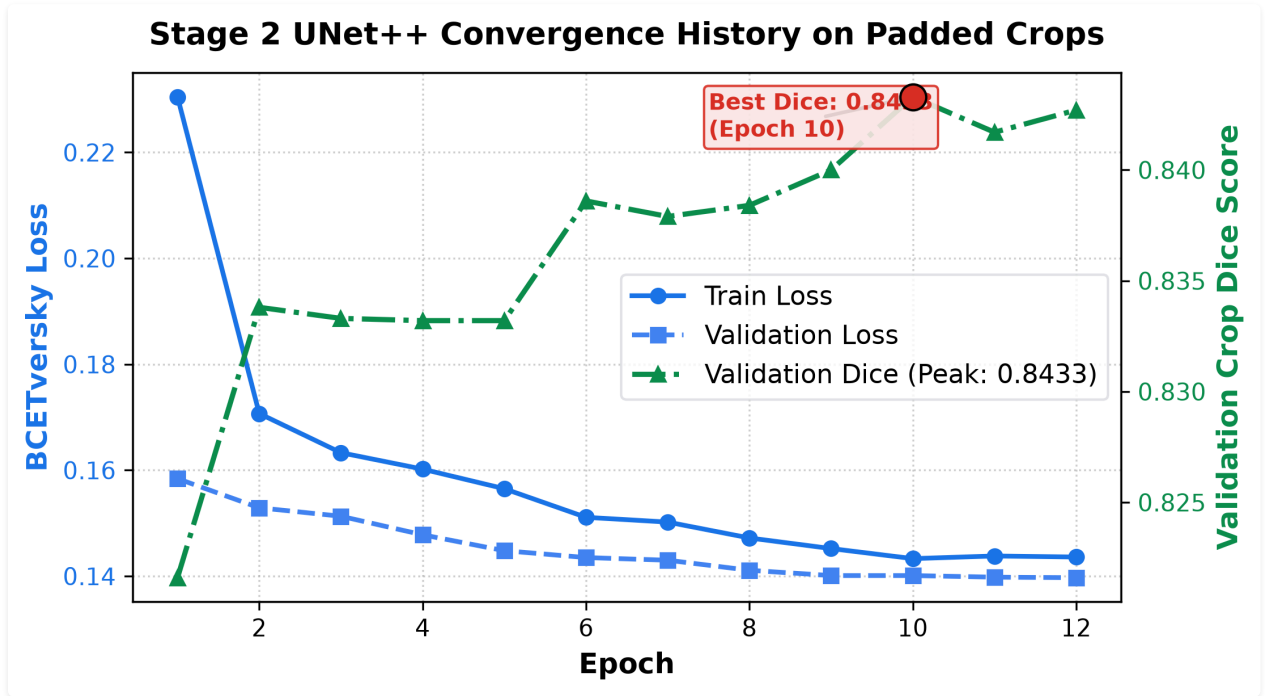


Figure 2: Stage 2 UNet++ convergence across 12 training epochs on padded crops, reaching a peak validation crop Dice coefficient of 0.8433 at epoch 10.

3.3 Greedy Panoptic Assembly & Algorithm

To strictly satisfy the competition's non-overlapping constraint (Eq. 2), predicted candidate masks are greedily painted in descending order of detection confidence:

```

def paint_panoptic(cached, shape, conf_t, mask_t, min_area=CFG.MIN_AREA):
    # In descending confidence order: each pixel goes to the first filament claiming it
    h, w = shape
    items = sorted([it for it in cached if it['conf'] >= conf_t], key=lambda d: -d['conf'])
    occupied = np.zeros((h, w), dtype=bool)
    masks = []
    for it in items:
        cx1, cy1, cx2, cy2 = it['box']
        m = np.zeros((h, w), dtype=bool)
        m[cy1:cy2, cx1:cx2] = it['probs'].astype('float32') > mask_t
        m &= ~occupied
        if m.sum() <= min_area:
            continue
        occupied |= m
        masks.append(m)
    return masks

```

Listing 3: Greedy panoptic assembly function ensuring non-overlapping instance masks.

Algorithm 1: Two-Stage Panoptic Filament Segmentation Inference

Require: Full-disk image $I \in \mathbb{R}^{2048 \times 2048}$, YOLO11m detector D , UNet++ segmenter S

Ensure: Disjoint instance masks $[M_1, \dots, M_N]$

- 1: $B \leftarrow D(I; \text{imgsz}=1024, \text{conf}=0.05, \text{max_det}=60)$
- 2: Sort B in descending order of detection confidence
- 3: **for** each box $b_k = (x, y, w, h, \text{conf}_k) \in B$ **do**
- 4: Compute padded crop coordinates (x_1, y_1, x_2, y_2) via Eq. 3
- 5: $C_k \leftarrow \text{Resize}(I[y_1:y_2, x_1:x_2], 512 \times 512)$
- 6: $P_k \leftarrow S(C_k)$ (Sigmoid probability map)
- 7: $P_k \leftarrow \text{ResizeBack}(P_k, (x_1, y_1, x_2, y_2))$
- 8: **end for**
- 9: $\text{claimed} \leftarrow \emptyset$
- 10: **for** $k = 1$ to N (descending confidence order) **do**
- 11: **if** $\text{conf}_k \geq 0.35$ **then**
- 12: $M_k \leftarrow (P_k > 0.60) \cap \neg \text{claimed}$
- 13: $\text{claimed} \leftarrow \text{claimed} \cup M_k$
- 14: **end if**
- 15: **end for**
- 16: **return** $[M_k : |M_k| > 0]$ encoded as compressed COCO RLE

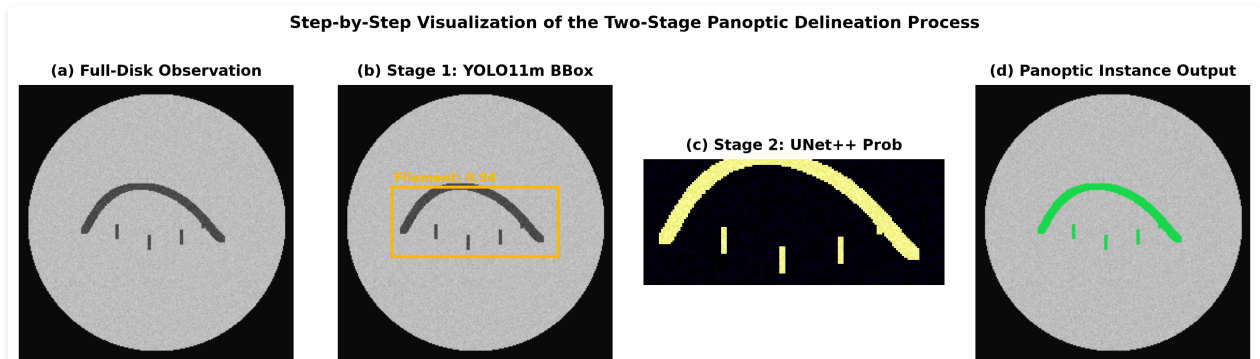


Figure 3: Visual step-by-step demonstration: (a) Full-disk observation; (b) Stage 1 YOLO11m candidate box; (c) Stage 2 UNet++ probability map; (d) Final panoptic instance segmentation overlay.

4. Experimental Results and Performance Analysis

4.1 Validation and Leaderboard Results

Experiments were conducted on a single NVIDIA Tesla T4 GPU (15 GB VRAM) using PyTorch 2.10 and CUDA 12.8. On the 107-image validation set, the pipeline records TP = 815, FP = 466, and FN = 579 matched instances at the IoU > 0.5 threshold.

Table 1: Quantitative Evaluation Results on the MAGFiLO 1.0 Dataset.

Metric / Pipeline	Detection Quality (DQ)	Segmentation Quality (SQ)	Panoptic Quality (PQ)	Crop Dice	Public Score
Human Inter-Annotator Agreement [1]	—	—	≈ 0.3600	—	—
Our Two-Stage Pipeline (V1)	0.6093	0.6802	0.4145	0.8433	0.3700

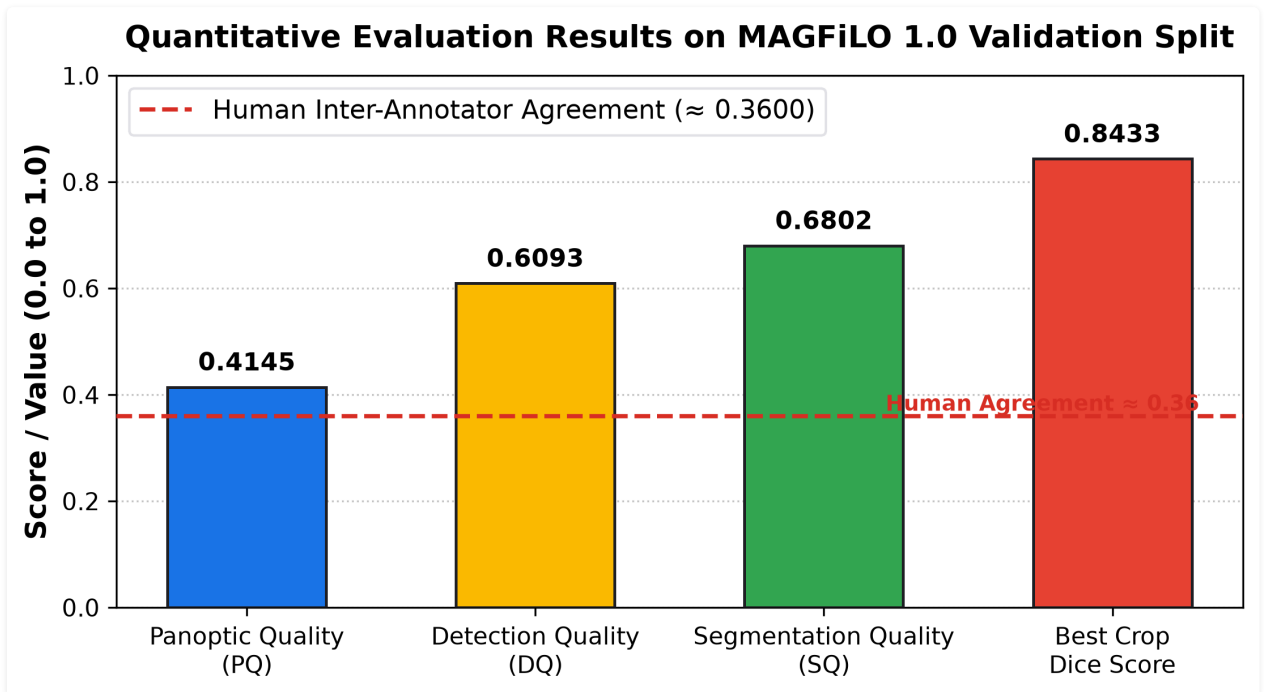


Figure 4: Quantitative validation metrics compared against the human inter-annotator agreement baseline (≈ 0.3600).

4.2 Discussion & Bottlenecks

The validation Detection Quality (0.6093) is the primary bottleneck relative to Segmentation Quality (0.6802): once a filament candidate is correctly localized and matched to ground truth, the crop-level segmenter delineates its boundary with high fidelity (crop Dice of 0.8433). This is consistent with the qualitative characterization of the task — faint, low-contrast filaments are more often missed entirely by the Stage-1 detector (contributing to FN) or localized imprecisely against adjacent structures (contributing to FP).

Notably, the achieved validation PQ of 0.4145 and public leaderboard score of 0.3700 exceed the reported human inter-annotator agreement benchmark of ≈ 0.36 [1], confirming that decoupled, two-stage detection-then-segmentation is an effective and computationally tractable strategy for automated solar filament segmentation.

5. Conclusion

This research presented an end-to-end, two-stage deep learning framework for automated instance segmentation of solar filaments in synoptic GONG H α imagery. By combining a YOLO11m candidate detector with a UNet++ (EfficientNet-B4) crop-level segmenter trained under a composite BCE–Tversky loss, the proposed method effectively addresses foreground–background class imbalance and diffuse boundary morphology. The greedy, confidence-ranked panoptic assembly procedure guarantees strictly disjoint instance masks, achieving a validation PQ of 0.4145 and a public leaderboard score of 0.3700.

References

1. A. Ahmadzadeh, et al., "A dataset of manually annotated filaments from H-alpha observations," *Nature Scientific Data*, vol. 11, p. 1031, 2024.
2. A. Ahmadzadeh, D. J. Kempton, Q. Li, and A. A. Pevtsov, "Solar Filament Segmentation Challenge 2026," *Kaggle Competition*, 2026.
3. G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
4. A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár, "Panoptic Segmentation," in *Proc. IEEE/CVF CVPR*, 2019, pp. 9404–9413.
5. S. S. M. Salehi, D. Erdogmus, and A. Gholipour, "Tversky Loss Function for Image Segmentation Using 3D Fully Convolutional Deep Networks," in *Proc. MLMI*, 2017, pp. 379–387.
6. M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. ICML*, 2019, pp. 6105–6114.
7. Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh, and J. Liang, "UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation," *IEEE TMI*, vol. 39, pp. 1856–1867, 2020.